Rainbow AI — build plan (v1, 2026-10-20)

Checklist edition for printing. Content is the plan as written; only the layout changed. Tick a step when its "Done when" test passes.

Companion docs: `claude/rainbow-ai-architecture.md` (how the pieces fit, technical reference) and `claude/rainbow-ai-glossary.md` (every term explained plainly). `claude/windows-port.md` covers what's already on the Windows box.

What we're building

A program on the Windows 11 shop computer ("RAINBOW-AI") that watches the shop Gmail, turns every email with files into a job folder with a **datasheet** (size, color/BW page counts, copies, stock, warnings), does the press-prep setup with the existing `tools/` scripts when needed, shows a **job board** in a browser on any shop computer, and **prints through Fiery hot folders only after a human clicks Print**. The price book lives inside the same program. Claude is the judgment layer (reading instructions, color/BW calls, choosing setup), scripts do everything deterministic, and nothing prints without approval.

Decisions locked in (2026-10-20)

- **Runtime Claude = Andrew's claude.ai subscription** via `claude setup-token` (one-year token). Revisit API billing later if usage becomes a problem. Runtime model: Sonnet.
- **Printing = Fiery hot folders**, one per common configuration. No print-dialog automation. Per-job copies: set at release in Command WorkStation at first; later JDF ticket, Fiery API, or one Windows print queue per preset (see architecture doc, "Copies").
- **Minimize computer use.** Daily loop uses zero browser/screen automation. Email digest reads red stars through Gmail search (`has:red-star`) — to be verified in step 1.2; headless browser only as a fallback for star colors and stubborn share links.
- **Color/BW is decided the way the shop prints it**, not the way the file is built: customer instruction → script metrics for clear cases → Claude looks at the pages for the middle band → always overridable in the UI by Andrew, Mark or Dave. Every override is logged and used to tune the rules.
- **Gmail via OAuth on the shop account** (Mark and Dave OK'd it), Drive API on the same consent.
- **Windows box is the master.** Price book is edited in the web app (nothing to sync). Code/skills/tools live in a private GitHub repo; the Mac is a second clone.
- **Everything in `tools/` stays as-is.** The app calls the scripts; it doesn't replace them.

How build sessions work (the contract)

1. **One step per task.** Open a new Claude task, say "Rainbow AI step 0.3" — Claude reads this plan, explains the step, then does it.
2. **Claude explains before and as it goes**: what the thing is, why this way, what the command does. Ask "why?" any time; it's part of the build.
3. **You only type what's listed under "You".** Commands come as copy-paste blocks for Git Bash on the Windows box. Claude writes all code.
4. **Every step ends with a visible test** ("Done when"). We don't move on until it passes.
5. **Build log.** Claude appends to `BUILD-LOG.md` in the repo after every step: what was done, what you learned, what's next. Future sessions read it first.
6. **Usage discipline.** Build sessions: Sonnet for routine coding, Opus/Fable for design decisions and when Sonnet gets stuck. No parallel subagents. Runtime: Sonnet with `--max-turns` caps.
7. **Git, minimal version.** You will use two commands: `git status` (what changed) and `git pull` (get the latest). Claude does `add/commit/push` and explains each commit message.

Phases and steps

Each step: **Goal · You · Claude · Done when · You'll learn.** Steps inside a phase go in order; phases can overlap a little (Phase 0.6/0.7 can happen while Phase 1 is in progress).

Phase 0 — Foundations (the Windows box becomes the workshop)

☐ 0.1 Confirm the Windows box state

Goal: Python, Git, poppler and the tools work natively on Windows.

You: Open Git Bash on RAINBOW-AI; paste `cd /c/Rainbow && bash tools/tests/run_tests.sh` (Claude gives the exact line).

Claude: Reads `tools/windows/setup_log.txt`, fixes anything missing.

Done when: tests print PASS and exit 0.

You'll learn: what a terminal is, what PATH means, why `py` vs `python3`.

☐ 0.2 Install Claude Code + VS Code on the Windows box

Goal: Claude Code runs natively on RAINBOW-AI, logged into your account; VS Code shows files, terminal and Claude side by side.

You: Install VS Code and the Claude Code extension; run `claude` once in `C:\Rainbow` and log in through the browser.

Claude: Checks `claude --version`, `/status`, confirms it sees `C:\Rainbow`.

Done when: `claude -p "list the scripts in tools/"` answers correctly.

You'll learn: Claude Code (terminal) vs Cowork (desktop app) vs the web; what `CLAUDE.md` is.

☐ 0.3 Create the GitHub repo — `C:\Rainbow` becomes the repo

Goal: Version control for code, tools, skills and price-book data; nothing else (job folders, customer files, secrets stay out).

You: Create a GitHub account (if none), create a private repo `rainbow-ai`; run the 3 commands Claude hands you (`git config` name/email, `git init`, `git remote add`).

Claude: Writes an allow-list `.gitignore` (only `tools/`, `PriceBook/` minus `private/`, `.claude/`, `app/`, `CLAUDE.md`, `BUILD-LOG.md` are tracked), makes the first commit, pushes.

Done when: the repo shows the tools on github.com and `git status` is clean.

You'll learn: repo, commit, push/pull, `.gitignore`, why secrets never go in git.

☐ 0.4 Put the Rainbow rules and skills into the repo

Goal: The runtime agent follows the same rules you've trained in Cowork.

You: Nothing to type; review what Claude writes.

Claude: Exports `rainbow-print-imposition`, `rainbow-photo-print-prep`, `rainbow-print-order-packet`, `rainbow-email-digest`, `rainbow-email-reply`, `rainbow-pricebook` into `C:\Rainbow\.claude\skills\`; turns the project instructions into `C:\Rainbow\CLAUDE.md`.

Done when: `claude -p "/rainbow-print-imposition make tools/card_source.pdf 2-up on letter with marks" --model sonnet` produces a verified PDF on Windows.

You'll learn: skills are folders of markdown; CLAUDE.md is the standing-orders file.

☐ 0.5 Runtime credential (subscription token)

Goal: Scripts can call Claude without a browser login.

You: Run `claude setup-token`, approve in the browser, paste the token into a user environment variable `CLAUDE_CODE_OAUTH_TOKEN` (Claude walks you through System Properties → Environment Variables; or one `setx` line).

Claude: Verifies from a fresh terminal: `claude -p "say ok" --model sonnet`.

Done when: it says ok with no login prompt.

You'll learn: environment variables, tokens, why `--bare` can't be used with this token (it only reads API keys).

☐ 0.6 Google OAuth for Gmail + Drive

Goal: The program can read the shop Gmail, apply labels, and download files from Drive links shared with the shop.

You: In Google Cloud Console (logged in as the shop account): create project "Rainbow AI", enable Gmail API and Drive API, configure the OAuth consent screen (External, then set publishing status to **In production** — otherwise the login expires every 7 days), create a Desktop OAuth client, download `credentials.json` to `C:\Rainbow\secrets\`. Run Claude's one-time `authorize.py` and click Allow.

Claude: Writes `authorize.py`, confirms a token file appears, lists the 5 newest threads as the test.

Done when: `py app/intake/gmail_test.py` prints 5 subjects.

You'll learn: OAuth = "let this program act as me without my password"; scopes; refresh tokens.

Plan B: if the consent screen fights us, Gmail IMAP with an App Password (needs 2-Step Verification on the shop account) is a 15-minute fallback for email; Drive private links would then go through the browser fallback.

☐ 0.7 Fiery hot folders for every common configuration

Goal: Each print configuration you use regularly is a hot folder with a preset, pointed at the **Held** queue.

You: In Fiery Hot Folders on the Windows box, create folders named by a convention Claude gives you (e.g. `V180_BW_SS_Letter_Plain`, `V180_BW_DS_Letter_Plain`, `V180_CLR_SS_Letter_Plain`, `V180_CLR_SS_Letter_Cardstock110`, `V180_BW_SS_SlipSheet`, …). Also check **Server Info** in CWS for the Fiery system version (FS150 or later matters for Phase 4).

Claude: Writes `app/presets.json` mapping each folder to its media/sides/color so the datasheet and the Print button can use them.

Done when: dropping a test PDF into a folder lands it in Held with the right properties.

You'll learn: what the Hot Folders app actually does (it watches a folder and submits with a saved preset); Held vs Print queue.

Phase 1 — Intake: email → job folder → datasheet (no UI yet)

☐ 1.1 The job database

Goal: One file, `C:\Rainbow\app\data\rainbow.sqlite`, holds every job, file, event and override.

Claude: Writes the schema (tables `jobs`, `files`, `events`, `overrides`, later `prices`), a tiny `db.py`, and a `dbview.py` that prints tables as text.

Done when: `py app/dbview.py jobs` prints an empty table with the right columns.

You'll learn: a database is a file of spreadsheets; what a schema, a row and a primary key are; why SQLite needs no server.

☐ 1.2 Gmail poller

Goal: New threads with attachments become job folders `C:\Rainbow\Jobs\YYYY-MM-DD_<customer>_<subject>\source\`.

Claude: Writes `poll_gmail.py`: lists threads since last poll, skips red-starred (**test** `has:red-star` **here** — if the API honors it, the digest never needs a browser), saves attachments, records the job, applies label `RainbowAI/Seen`. Never touches stars.

Done when: you send yourself a test email with a PDF and a folder appears within a minute of running the script.

You'll learn: polling, APIs, JSON, labels vs stars.

☐ 1.3 Share-link fetching

Goal: Links in the email body become files in `source/`.

Claude: Wires in `tools/fetch_links.py` (OneDrive works today), adds Drive API for private shares, Dropbox `dl=1`, and a headless-Chromium fallback (Playwright) for WeTransfer and friends. Failure → job status `needs_file` with the link listed.

Done when: a OneDrive link, a Drive link shared with the shop, and a Dropbox link all land as files.

You'll learn: why a share link is a web page, not a file; headless browsers.

☐ 1.4 Preflight + color metrics

Goal: Deterministic facts about every file before any judgment.

Claude: Runs `pdfinspect.py` per file; converts images/Office files to PDF (LibreOffice is installed); writes `colorpages.py` — per page: color coverage %, largest color blob %, text-only-color flag.

Done when: a logo-only letter and a photo flyer get obviously different numbers.

You'll learn: how "is this page color?" is actually measured.

☐ 1.5 Spec extraction (Claude reads the email)

Goal: Customer text + preflight → a `spec.json` (copies, sides, size, stock, color intent, finishing, due date, ambiguities, questions).

Claude: One `claude -p --output-format json --json-schema …` call on Sonnet with the email text, the metrics and page thumbnails for the undecided color cases.

Done when: 10 real past emails produce specs you'd sign.

You'll learn: structured output; prompts are instructions + a form to fill in.

☐ 1.6 Color/BW decision rules

Goal: The shop's judgment, encoded: customer says → that; big color areas → color; logo-only → BW; middle band → Claude looks; everything logged with its reason.

Done when: 20 past jobs match what you actually did ≥ 18 times; thresholds live in `app/color_rules.json`.

You'll learn: rule-first, model-second; how overrides become training data.

☐ 1.7 Datasheet

Goal: `datasheet.pdf` + `datasheet.html` in every job folder.

Done when: you'd hand it to Mark as the job ticket.

☐ 1.8 Run it automatically

Goal: The poller runs every 2 minutes whether or not anyone is logged in.

You: Approve the scheduled task / service install.

Done when: a week of real emails has produced folders and datasheets without you touching it.

☐ **Checkpoint 1: triage labor is gone.**

Phase 2 — Job board (the web interface)

☐ 2.1 Hello, server

Goal: A web page served by RAINBOW-AI opens from a counter PC at `http://rainbow-ai:8000`.

You: Click Allow on the Windows Firewall prompt; open the address on another PC.

You'll learn: web server, port, localhost vs LAN, firewall.

☐ 2.2 Job list

Columns: New · Needs file · Needs setup · Ready · Sent to Fiery · Printed · Done. Auto-refresh. Search by customer.

☐ 2.3 Job page

Datasheet, page thumbnails, **Color/BW toggle**, copies, stock dropdown (from `presets.json`), corrections box, event history. Overrides write to `overrides`.

☐ 2.4 Inbox page (the email digest, live)

Every thread in the window, status (open / needs reply / done = red star), who at the shop replied last, linked job. Replaces the digest PDF.

☐ 2.5 Run as a service

Server starts at boot (NSSM). Health page: last poll, errors, Fiery reachable.

- [ ] **2.6 Tailscale (optional)**

  Same page from your MacBook at home.

  - [ ] **Checkpoint 2: Mark can open the board and see today's jobs.**

Phase 3 — Setup button (the agent)

- [ ] **3.1 Agent runner**

  `claude -p` with the rainbow skills, working directory = job folder, tools limited to Read + the `tools/` scripts, output to `PRINT READY/`, must finish with `verify_print.py` PASS. Previews regenerate. Usage per run logged.

- [ ] **3.2 Corrections box**

  Text → agent re-run with the correction, session resumed so it remembers. Few words needed ("2-up, no marks").

- [ ] **3.3 Guardrails**

  Never scale/crop (the scripts already refuse). Crush-guard lifts shown as a banner on the job page. verify FAIL → `setup_failed` with the log. `--max-turns` and a time limit per run.

  - [ ] **Checkpoint 3: "4x6 on cardstock, 2-up" goes from email to a verified, previewed PDF with one click.**

Phase 4 — Print

- [ ] **4.1 Print button + confirmation**

  Modal shows preview + readout: size, sides, color mode, copies, stock, hot folder. **Print** copies the verified PDF into that hot folder → job appears in Held. Status `sent_to_fiery`.

- [ ] **4.2 Copies without Command WorkStation**

  Pick one, after checking the Fiery version: JDF ticket, Fiery API, or one Windows print queue per preset driven by `SumatraPDF -print-to`. Architecture doc has the trade-offs.

- [ ] **4.3 Printed / Done**

  Mark Printed in the app → label `RainbowAI/Printed` on the thread. Red stars stay the owners' mark; the app only reads them.

  - [ ] **Checkpoint 4: click Print, walk to the Versant.**

Phase 5 — Price book in the app

- [ ] **5.1 Import and lookup**

  Import `PriceBook/` into the `prices` table; lookup page with search.

- [ ] **5.2 Edit page**

  Edit page with a change log (who, when, old → new). Counter sheets regenerate from the table with the existing sheet scripts.

- [ ] **5.3 Quote**

  Product + qty → price; "Add to job" puts the price on the datasheet; free-text quotes answered by Claude with the price table as context.

  - [ ] **Checkpoint 5: change a price in the browser on the Mac, it's live on every PC and on the next counter sheet.**

Phase 6 — Hardening and the learning loop

- [ ] **6.1 Monthly override review**
  Monthly override review → tune `color_rules.json`.
- [ ] **6.2 Backups**
  Nightly backup of `rainbow.sqlite` + `Jobs/` to shop storage.
- [ ] **6.3 Mac clone**
  Mac clone of the repo; `git pull` habit; Claude commits from either machine.
- [ ] **6.4 Logging**
  Logging, error emails to you, a "what broke" page.

Feature backlog (suggestions — pick what you want, in any order)

- [ ] **Ask-the-customer button:** when the spec has questions, draft the reply (rainbow-email-reply rules), you approve, it sends from the shop Gmail and the job waits in `awaiting_customer`.
- [ ] **Paper traveler:** print the datasheet through the slip-sheet hot folder so each physical job carries its ticket.
- [ ] **Walk-in / USB / scan intake:** drop a file on the web page (or into the Fiery "Scans" folder) and it enters the same pipeline as an email job.
- [ ] **"Same as last time":** customer history from the DB; one click to reuse a previous setup.
- [ ] **Auto-quote on the datasheet** from the price book (impressions × click rates + stock + finishing).
- [ ] **Preflight warnings on the datasheet:** unembedded fonts, low dpi, RGB rich black, crush-guard lifts, size mismatch.
- [ ] **Due-date board** and an 8 am summary.
- [ ] **Reprint button** for repeat jobs; **monthly stats** (jobs, color vs BW impressions, overrides).
- [ ] **Address book refresh** from the jobs DB (replaces the AddressBook kit).

Open questions

- [ ] Fiery system version (FS150+?) → decides JDF/API vs print-queue route for copies.
- [ ] Windows 11 Home or Pro on RAINBOW-AI (only matters for Remote Desktop; the web UI removes the need).
- [ ] Does the Gmail API honor `has:red-star` in `q`? (Step 1.2 test.)
- [ ] Which hot-folder configurations are "common" — your list in 0.7.